CETAM – Centro de Educação Tecnológica do Amazonas

Qualificação Profissional em Linguagem de Programação Python – 80 h

PROJETOS INTEGRADORES

Volume 2 – Projetos 6 a 10

Enunciados para os alunos e Gabarito Oficial do professor

Agência Amazônia Tur Digital – Projeto integrador do curso

Profa. Priscila Gonçalves – Manaus/AM

Sumário

Sumário 2
Apresentação 5
Visão geral dos projetos 5
Como desenvolver 5
Regras gerais de entrega 6
Modelo do quadro de testes 6
Projeto 6 – Quiz da Floresta Amazônica 7
Cenário 7
Objetivos de aprendizagem 7
Dados de partida 7
Requisitos funcionais 8
Regras de negócio 8
Exemplo de execução 8
O que entregar 9
Critérios de avaliação 9
Dicas 9
Desafios extras (opcionais) 9
Projeto 7 – Meu Orçamento Familiar 10
Cenário 10
Objetivos de aprendizagem 10
Dados de partida 10
Requisitos funcionais 11
Regras de negócio 11
Exemplo de execução 11
O que entregar 12
Critérios de avaliação 12
Dicas 12
Desafios extras (opcionais) 12
Projeto 8 – Forca Amazônica 14
Cenário 14
Objetivos de aprendizagem 14
Dados de partida 14
Requisitos funcionais 14
Regras de negócio 15
Exemplo de execução 15
O que entregar 16

Critérios de avaliação 16
Dicas 16
Desafios extras (opcionais) 16
Projeto 9 – Agenda de Contatos Comunitária 17
Cenário 17
Objetivos de aprendizagem 17
Dados de partida 17
Requisitos funcionais 17
Regras de negócio 18
Exemplo de execução 18
O que entregar 19
Critérios de avaliação 19
Dicas 19
Desafios extras (opcionais) 19
Projeto 10 – Biblioteca Comunitária 20
Cenário 20
Objetivos de aprendizagem 20
Dados de partida 20
Requisitos funcionais 21
Regras de negócio 21
Exemplo de execução 21
O que entregar 22
Critérios de avaliação 22
Dicas 22
Desafios extras (opcionais) 23
Gabarito Oficial dos Projetos Integradores 24
Como corrigir 24
Gabarito – Projeto 6: Quiz da Floresta Amazônica 25
Solução de referência 25
Execução de teste 26
Casos de teste para correção 30
Matriz de correção 30
Erros frequentes e orientações 31
Orientações para os desafios extras 31
Gabarito – Projeto 7: Meu Orçamento Familiar 32
Solução de referência 32
Execução de teste 33
Casos de teste para correção 39

Matriz de correção 39
Erros frequentes e orientações 40
Orientações para os desafios extras 40
Gabarito – Projeto 8: Forca Amazônica 41
Solução de referência 41
Execução de teste 42
Casos de teste para correção 45
Matriz de correção 45
Erros frequentes e orientações 45
Orientações para os desafios extras 46
Gabarito – Projeto 9: Agenda de Contatos Comunitária 47
Solução de referência 47
Execução de teste 50
Casos de teste para correção 54
Matriz de correção 54
Erros frequentes e orientações 54
Orientações para os desafios extras 55
Gabarito – Projeto 10: Biblioteca Comunitária 56
Solução de referência 56
Execução de teste 59
Casos de teste para correção 63
Matriz de correção 64
Erros frequentes e orientações 64
Orientações para os desafios extras 64

Apresentação

Este é o **Volume 2** dos Projetos Integradores do curso de Python. Ele traz **cinco projetos novos** (6 a 10), diferentes dos cinco do Volume 1, para que você aplique, de forma integrada, tudo o que praticou nas aulas: variáveis, decisões, repetições, listas, dicionários, strings, funções, arquivos de texto e orientação a objetos básica.

Visão geral dos projetos

Nº	Projeto	Nível	Tempo	Conteúdos principais
6	Quiz da Floresta Amazônica – Jogo de perguntas e respostas com pontuação	★☆☆☆☆	4 a 6 horas	variáveis, `input()`, operadores, `if/elif/else`, `while`, `for/range`, contadores e acumuladores
7	Meu Orçamento Familiar – Controle de receitas e despesas por categoria	★★☆☆☆	6 a 8 horas	listas, dicionários, listas de dicionários, strings, `for/range`, decisões e validações
8	Forca Amazônica – Jogo da forca com palavras da fauna e da flora regionais	★★★☆☆	8 a 10 horas	funções com parâmetros e retorno, tuplas, listas, strings, laços e decisões
9	Agenda de Contatos Comunitária – Cadastro com busca, edição, remoção e persistência	★★★★☆	10 a 12 horas	funções, listas de dicionários, strings, validações, arquivos `.txt`, busca, edição e remoção
10	Biblioteca Comunitária – Sistema de empréstimos orientado a objetos	★★★★★	12 a 16 horas	classes e objetos que colaboram entre si, dicionário de objetos, arquivos, validações e regras de negócio

Os projetos são **progressivos**: o 6 usa apenas os conteúdos iniciais; o 10 reúne classes, dicionários e persistência em arquivo. Você pode fazê-los na ordem ou escolher conforme o ponto do curso em que estiver. Para saber quais aulas cada projeto exige, veja o quadro de cada enunciado.

Como desenvolver

- **Leia o enunciado inteiro** antes de escrever a primeira linha de código, e grife os requisitos (RF) e as regras de negócio (RN).
- **Planeje no papel:** liste as entradas, os cálculos e as saídas. Se quiser, escreva um fluxograma simples.
- **Programe em etapas pequenas** e teste cada etapa. Não deixe para testar tudo no final.
- **Use VS Code ou Google Colab**, como preferir. Entregue o arquivo `.py` ou o notebook `.ipynb`, com os nomes dos integrantes em comentário no topo.
- **Compare seu resultado com o "Exemplo de execução"** do enunciado e monte seu quadro de testes.

- Todos os dados e valores usados são **fictícios** e servem apenas para o estudo.

Regras gerais de entrega

- O código deve rodar do início ao fim, sem erros, nos casos de teste do quadro.
- Use nomes de variáveis claros, comentários objetivos e `f-strings` para formatar valores (`:.2f` para dinheiro).
- Mensagens ao usuário em português, claras e educadas.
- A nota é atribuída exclusivamente pelo professor, conforme os critérios de avaliação de cada projeto.
- O trabalho é seu: pode e deve conversar com colegas, mas **não copie código** pronto.

Modelo do quadro de testes

Entregue um quadro como o abaixo, com no mínimo os casos pedidos em cada projeto:

Nº	Entrada / situação	Resultado esperado	Resultado obtido	OK?
1	Resposta “x” no quiz	Aviso de inválida e repete a pergunta		
2				

Projeto 6 – Quiz da Floresta Amazônica

Ficha do projeto	
Tema	Jogo de perguntas e respostas com pontuação
Nível	★☆☆☆☆
Tempo sugerido	4 a 6 horas
Forma de trabalho	individual ou dupla
Aulas relacionadas	Aulas 1 a 10
Conteúdos	variáveis, `input()`, operadores, `if/elif/else`, `while`, `for/range`, contadores e acumuladores

Cenário

A secretaria de educação de um município do Amazonas quer um **jogo de perguntas sobre a Amazônia** para usar nas escolas. Você vai criar o **Quiz da Floresta Amazônica**: o jogador responde a 5 perguntas de múltipla escolha, ganha pontos por acerto e **bônus por sequência de acertos**, recebe um título ao final e pode jogar várias partidas, disputando o recorde da sessão.

Objetivos de aprendizagem

Ao concluir este projeto, você será capaz de:

- Usar `for` com `range()` para percorrer as perguntas.
- Aplicar `if/elif/else` para escolher a pergunta e avaliar a resposta.
- Validar entradas com `while`.
- Usar **contadores e acumuladores** (acertos, pontos e sequência).
- Controlar um menu com repetição e `break`.

Dados de partida

Nº	Pergunta	Alternativas	Correta
1	Quais rios formam o Encontro das Águas, em Manaus?	A) Madeira e Purus B) Negro e Solimões C) Tapajós e Xingu D) Juruá e Japurá	B
2	Qual peixe amazônico pode ultrapassar 2 metros de comprimento?	A) Tambaqui B) Pirarucu C) Jaraqui D) Matrinxã	B
3	O Festival Folclórico de Parintins reúne quais bois-bumbás?	A) Garantido e Caprichoso B) Mimoso e Caprichoso C) Garantido e Mimoso D) Faceiro e Caprichoso	A
4	Em qual cidade fica o Teatro Amazonas?	A) Belém B) Manaus C) Parintins D) Santarém	B
5	Qual é o rio de maior volume de água do mundo?	A) Nilo B) Mississippi C) Amazonas D) Yangtzé	C

Requisitos funcionais

Código	Requisito
RF01	**Menu principal**: 1 – Jogar, 2 – Regras, 0 – Sair, repetido até o jogador escolher sair.
RF02	Em cada partida, apresentar **5 perguntas** (use `for numero in range(1, 6)` e `if/elif` para definir texto, alternativas e resposta correta).
RF03	Ler a resposta do jogador, **validar** (A, B, C ou D) e dizer se acertou; se errou, informar a alternativa correta.
RF04	Calcular a **pontuação** com o bônus de sequência (regras RN02 e RN03).
RF05	Ao fim da partida, mostrar **acertos, pontuação e título** do jogador.
RF06	Guardar o **recorde da sessão** e a **quantidade de partidas**; avisar quando houver novo recorde.
RF07	Ao sair, mostrar o total de partidas e a melhor pontuação.

Regras de negócio

Código	Regra
RN01	A resposta é aceita em **maiúscula ou minúscula**, com ou sem espaços ao redor. Resposta fora de A–D: avisar e **perguntar de novo**, sem penalidade.
RN02	Cada acerto vale **10 pontos**; erro vale 0.
RN03	**Bônus de +5 pontos a cada 3 acertos seguidos**. Depois do bônus, a contagem de acertos seguidos **recomeça do zero**. Qualquer erro também zera a contagem.
RN04	Título: **50 pontos ou mais**, “Guardião da Floresta”; **de 30 a 49**, “Explorador”; **abaixo de 30**, “Aprendiz”.
RN05	Há **novo recorde** quando a pontuação da partida é **maior** que a melhor da sessão.
RN06	Opção de menu inexistente: “Opção inválida!”.

Exemplo de execução

Trecho de uma execução do programa (o que o usuário digita aparece depois dos dois-pontos de cada pergunta):

Saída esperada

```
=== QUIZ DA FLORESTA AMAZÔNICA ===
1 - Jogar
2 - Regras
0 - Sair
Escolha: 2

--- REGRAS ---
São 5 perguntas de múltipla escolha.
Cada acerto vale 10 pontos.
A cada 3 acertos seguidos, você ganha 5 pontos de bônus.
50+ pontos: Guardião da Floresta | 30 a 49: Explorador | abaixo de 30: Aprendiz.

=== QUIZ DA FLORESTA AMAZÔNICA ===
1 - Jogar
2 - Regras
```

```
0 - Sair
Escolha: 1

Pergunta 1 de 5
Quais rios formam o Encontro das Águas, em Manaus?
A) Madeira e Purus
B) Negro e Solimões
C) Tapajós e Xingu
D) Juruá e Japurá
Sua resposta (A, B, C ou D): B
Correto! +10 pontos.

Pergunta 2 de 5
Qual peixe amazônico pode ultrapassar 2 metros de comprimento?
```

O que entregar

- Arquivo `projeto06_quiz.py` (VS Code) **ou** notebook `Projeto06_Quiz.ipynb` (Colab), com os nomes dos integrantes em comentário no topo.
- Quadro de **testes** com, no mínimo, 5 casos (inclua resposta inválida, sequência de 3 acertos e partida sem acertos).

Critérios de avaliação

Critério	O que será observado
Funcionamento	O jogo roda do menu ao encerramento, com várias partidas, sem travar.
Regras de pontuação	Pontos, bônus de sequência, títulos e recorde corretos.
Uso dos conteúdos	`for/range`, `if/elif/else`, `while`, contadores e acumuladores.
Validação	Respostas e opções inválidas tratadas conforme as regras.
Organização e testes	Código legível, comentado e quadro de testes coerente.

Dicas

- Zere `pontos`, `acertos` e `seguidos` **no início de cada partida**.
- Para aceitar “b”, “ B ” ou “B”, use `.strip().upper()` na resposta.
- Valide com `while resposta != "A" and resposta != "B" and ...` (cuidado: com `or` a condição seria sempre verdadeira).
- Quando `seguidos == 3`, some o bônus e **volte `seguidos` para 0**.
- Atualize o recorde **depois** de calcular a pontuação final.

Desafios extras (opcionais)

- **Mais perguntas:** quando estudar listas, guarde as perguntas em uma lista de dicionários.
- **Dificuldade:** crie um multiplicador de pontos (fácil ×1, difícil ×2).
- **Ajuda:** uma vez por partida, elimine duas alternativas erradas (e perca 5 pontos).

Projeto 7 – Meu Orçamento Familiar

Ficha do projeto	
Tema	Controle de receitas e despesas por categoria
Nível	★★☆☆☆
Tempo sugerido	6 a 8 horas
Forma de trabalho	individual ou dupla
Aulas relacionadas	Aulas 1 a 14
Conteúdos	listas, dicionários, listas de dicionários, strings, `for`/`range`, decisões e validações

Cenário

Muitas famílias não sabem **para onde vai o dinheiro** no fim do mês. Você vai criar o **Meu Orçamento Familiar**, um programa em que a pessoa registra suas **receitas** e **despesas por categoria**, e o sistema mostra o **saldo**, a **porcentagem da renda comprometida**, o **gasto por categoria** e qual categoria pesa mais.

Todos os valores usados nos testes são **fictícios**.

Objetivos de aprendizagem

Ao concluir este projeto, você será capaz de:

- Guardar lançamentos em uma **lista de dicionários**.
- Usar uma **lista de categorias** e um **dicionário** de totais.
- Percorrer listas com `for` e `range(len(...))`.
- Calcular totais, saldos e porcentagens.
- Padronizar textos e validar entradas.

Dados de partida

Nº	Categoria de despesa	Exemplo
1	Moradia	aluguel, energia, água
2	Alimentação	feira, mercado
3	Transporte	barco, ônibus, combustível
4	Saúde	consulta, remédio
5	Lazer	cinema, passeio
6	Outros	demais gastos

Requisitos funcionais

Código	Requisito
RF01	**Menu**: 1 – Registrar receita, 2 – Registrar despesa, 3 – Resumo do mês, 4 – Despesas por categoria, 5 – Listar lançamentos, 0 – Sair.
RF02	**Receita**: pedir descrição e valor e guardar o lançamento (tipo “receita”).
RF03	**Despesa**: pedir descrição, **categoria (pelo número)** e valor e guardar o lançamento (tipo “despesa”).
RF04	**Resumo**: total de receitas, total de despesas, saldo, **porcentagem da renda comprometida** e a situação do orçamento.
RF05	**Despesas por categoria**: total de cada categoria com despesas, a **porcentagem** em relação ao total de despesas e a **categoria de maior gasto**.
RF06	**Listar** todos os lançamentos, numerados, com categoria, descrição, sinal (+ ou −) e valor.

Regras de negócio

Código	Regra
RN01	A descrição não pode ser vazia; padronize com `strip()` e `capitalize()`.
RN02	O valor deve ser **maior que zero**; caso contrário, pedir de novo.
RN03	A categoria deve ser um número de **1 a 6**; caso contrário, pedir de novo. Receitas usam a categoria “Receita”.
RN04	Comprometimento = despesas ÷ receitas × 100. **Até 70%**: “Saudável”. **Mais de 70% até 100%**: “Atenção – despesas altas”. **Acima de 100%**: “Alerta – despesas maiores que a receita”.
RN05	Sem receitas registradas, o resumo mostra os totais e avisa “Registre ao menos uma receita para analisar o orçamento.” (sem dividir por zero).
RN06	Sem despesas ou sem lançamentos, as opções 4 e 5 avisam, sem gerar erro.

Exemplo de execução

Trecho de uma execução do programa (o que o usuário digita aparece depois dos dois-pontos de cada pergunta):

Saída esperada

```
=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 3

--- RESUMO DO MÊS ---
Receitas: R$ 0.00
Despesas: R$ 0.00
Saldo: R$ 0.00
Registre ao menos uma receita para analisar o orçamento.
```

```
=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 4
Nenhuma despesa registrada.

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
```

O que entregar

- Arquivo `projeto07_orcamento.py` ou notebook `Projeto07_Orcamento.ipynb`, com nomes dos integrantes no topo.
- Quadro de **testes** com, no mínimo, 6 casos (inclua os três níveis de situação do orçamento e os casos "sem dados").

Critérios de avaliação

Critério	O que será observado
Funcionamento	Todas as opções do menu funcionam com os dados de teste.
Estruturas de dados	Lista de dicionários, lista de categorias e dicionário de totais.
Cálculos	Totais, saldo e porcentagens corretos.
Regras de negócio	RN01 a RN06 implementadas.
Organização e testes	Código legível, comentado e quadro de testes coerente.

Dicas

- Cada lançamento: `{"tipo": "despesa", "descricao": "Aluguel", "categoria": "Moradia", "valor": 1200.0}`.
- Mostre as categorias com `for i in range(len(categorias)):` e use `categorias[numero - 1]`.
- Crie o dicionário de totais zerado com um `for` sobre `categorias` e some com `totais[categoria] += valor`.
- Para a categoria de maior gasto, guarde `maior_valor` e `maior_categoria` enquanto percorre.
- Teste `receitas == 0` **antes** de dividir.

Desafios extras (opcionais)

- **Gráfico de barras:** mostre `"#" * int(percentual / 5)` ao lado de cada categoria.
- **Meta de poupança:** peça uma meta e informe se o saldo a atingiu.

- **Remover lançamento:** escolha pelo número da listagem.
- **Persistência:** salve os lançamentos em arquivo (estude a Aula 17).

Projeto 8 – Forca Amazônica

Ficha do projeto	
Tema	Jogo da forca com palavras da fauna e da flora regionais
Nível	★★★☆☆
Tempo sugerido	8 a 10 horas
Forma de trabalho	dupla
Aulas relacionadas	Aulas 1 a 16
Conteúdos	funções com parâmetros e retorno, tuplas, listas, strings, laços e decisões

Cenário

Uma escola quer um **jogo da forca** para treinar o vocabulário sobre a **fauna e a flora da Amazônia**. O jogador tem **6 vidas**, tenta descobrir a palavra letra por letra e acumula pontos ao longo das rodadas.

O código deve ser organizado em **funções**, cada uma com uma responsabilidade clara.

Objetivos de aprendizagem

Ao concluir este projeto, você será capaz de:

- Criar e combinar **funções** com parâmetros e retorno (inclusive retornando mais de um valor).
- Usar **tuplas** (palavra e dica) e **listas** (letras acertadas e erradas).
- Manipular **strings** (`lower`, `strip`, `isalpha`, percorrer letras).
- Controlar rodadas com `while` e pontuação acumulada.

Dados de partida

Rodada	Palavra	Dica
1	tambaqui	Peixe de escamas, famoso na culinária do Amazonas
2	pirarucu	Um dos maiores peixes de água doce, pode passar de 2 metros
3	tucunare	Peixe de rios e lagos, muito procurado na pesca esportiva
4	castanha	Semente de árvore gigante, também chamada castanha-do-pará
5	cupuacu	Fruta de polpa cremosa, usada em sucos e doces
6	jaraqui	Peixe pequeno e muito consumido pelos manauaras

Requisitos funcionais

Código	Requisito
RF01	Guardar as palavras e dicas em uma **tupla de tuplas** `PALAVRAS` (sem acentos, como na tabela).
RF02	Função `escolher_palavra(numero_rodada)`: devolve a palavra e a dica da rodada (a rodada 7 volta à palavra da rodada 1).

Código	Requisito
RF03	Função `montar_exibicao(palavra, acertadas)`: devolve o texto da palavra com _ nas letras ainda não descobertas (ex.: `t a _ b _ q _ i`).
RF04	Função `venceu(palavra, acertadas)`: devolve `True` se todas as letras foram descobertas.
RF05	Função `ler_letra(tentadas)`: lê e valida uma letra, repetindo a pergunta até receber uma entrada válida.
RF06	Função `jogar_rodada(numero_rodada)`: executa uma rodada e devolve `(ganhou, pontos)`.
RF07	Função `principal()`: controla as rodadas, mantém o **placar** (pontos e vitórias) e pergunta "Jogar outra rodada? (s/n)".

Regras de negócio

Código	Regra
RN01	O jogador começa cada rodada com **6 vidas**; cada letra errada tira uma vida. Com 0 vidas, a rodada termina em derrota.
RN02	A letra digitada é padronizada com `strip()` e `lower()`. Só vale **uma letra** de "a" a "z".
RN03	Letra **já tentada** (certa ou errada): avisar e pedir outra, **sem perder vida**.
RN04	Vitória: pontos da rodada = **10 + 5 × vidas restantes**. Derrota: **0 ponto**.
RN05	A cada rodada, mostrar a dica, a palavra parcial, as vidas e as letras erradas.
RN06	Ao encerrar, mostrar o número de rodadas, vitórias e pontuação total.

Exemplo de execução

Trecho de uma execução do programa (o que o usuário digita aparece depois dos dois-pontos de cada pergunta):

Saída esperada

```
=== RODADA 1 ===
Dica: Peixe de escamas, famoso na culinária do Amazonas

Palavra: _ _ _ _ _ _ _ _
Vidas: O O O O O O
Digite uma letra: t
Acertou!

Palavra: t _ _ _ _ _ _ _
Vidas: O O O O O O
Digite uma letra: x
Errou!

Palavra: t _ _ _ _ _ _ _
Vidas: O O O O O X
Letras erradas: ['x']
Digite uma letra: t
Você já tentou essa letra!
Digite uma letra: 12
Digite apenas UMA letra (de a a z).
```

```
Digite uma letra: ab
Digite apenas UMA letra (de a a z).
Digite uma letra: a
Acertou!

Palavra: t a _ _ a _ _ _
Vidas: O O O O O X
Letras erradas: ['x']
Digite uma letra: m
```

O que entregar

- Arquivo `projeto08_forca.py` ou notebook `Projeto08_Forca.ipynb`, com nomes da dupla no topo.
- Quadro de **testes** com, no mínimo, 6 casos e **registro de colaboração** (quem fez cada função).

Critérios de avaliação

Critério	O que será observado
Funcionamento	Rodadas completas (vitória e derrota) e placar corretos.
Modularização	Funções com responsabilidade única, parâmetros e `return` adequados.
Strings e listas	Letras certas/erradas controladas; exibição correta.
Regras de negócio	RN01 a RN06 implementadas.
Organização e colaboração	Código legível, testes e registro da dupla.

Dicas

- `for letra in palavra:` percorre as letras de um texto.
- Mantenha duas listas: `acertadas` e `erradas`. A lista de **tentadas** é `acertadas + erradas`.
- Uma função pode devolver dois valores: `return True, pontos` — e quem chama recebe com `ganhou, pontos = jogar_rodada(1)`.
- Para mostrar as vidas: `"O " * vidas + "X " * len(erradas)`.
- Com a tupla `(palavra, dica)`, use `palavra, dica = escolher_palavra(numero)`.

Desafios extras (opcionais)

- **Desenho da forca:** crie uma tupla com 7 desenhos em ASCII e mostre o estágio de acordo com os erros.
- **Palavras sorteadas:** use `import random` e `random.choice(PALAVRAS)`.
- **Placar em arquivo:** grave o recorde de pontos em `placar.txt`.
- **Mais categorias:** organize palavras por tema em um dicionário.

Projeto 9 – Agenda de Contatos Comunitária

Ficha do projeto	
Tema	Cadastro com busca, edição, remoção e persistência
Nível	★★★★☆
Tempo sugerido	10 a 12 horas
Forma de trabalho	dupla
Aulas relacionadas	Aulas 1 a 17
Conteúdos	funções, listas de dicionários, strings, validações, arquivos `.txt`, busca, edição e remoção

Cenário

A associação de moradores de um bairro de Manaus quer uma **agenda de contatos** digital para localizar rapidamente vizinhos, comerciantes e serviços. O sistema deve permitir **cadastrar, listar em ordem alfabética, buscar por parte do nome, editar e remover** contatos, e **guardar tudo em arquivo**.

Use apenas dados **fictícios** nos testes.

Objetivos de aprendizagem

Ao concluir este projeto, você será capaz de:

- Implementar as operações de um **CRUD** (criar, ler, atualizar e remover).
- Persistir uma lista de dicionários em **arquivo `.txt`**.
- Validar telefone e e-mail com funções de **string**.
- Organizar o sistema em **funções**.

Dados de partida

Campo	Exemplo	Regra
nome	Maria Das Dores	obrigatório, único, padronizado com `title()`
telefone	92991234567	só dígitos, 10 ou 11 (com DDD)
email	maria@email.com	contém “@” e “.” e não tem espaços
bairro	Centro	opcional; se vazio, “Não informado”

Requisitos funcionais

Código	Requisito
RF01	Ao iniciar, **carregar** os contatos de `agenda.txt` (arquivo inexistente = agenda vazia).
RF02	**Menu**: 1 – Cadastrar, 2 – Listar, 3 – Buscar por trecho do nome, 4 – Editar, 5 – Remover, 6 – Resumo por bairro, 0 – Sair.

Código	Requisito
RF03	**Cadastrar** contato (nome, telefone, e-mail e bairro) e salvar no arquivo.
RF04	**Listar** todos os contatos em **ordem alfabética** pelo nome.
RF05	**Buscar** contatos cujo nome **contenha** o trecho digitado, sem diferenciar maiúsculas de minúsculas.
RF06	**Editar** telefone, e-mail e bairro de um contato: **Enter em branco mantém** o valor atual. Salvar após a edição.
RF07	**Remover** um contato após **confirmação** (s/n) e salvar.
RF08	**Resumo por bairro**: quantos contatos há em cada bairro (use um dicionário contador).

Regras de negócio

Código	Regra
RN01	Nome padronizado com `strip()` e `title()`; **não vazio, sem ";" e sem repetição** na agenda.
RN02	Telefone: remover espaços, hífens e parênteses; aceitar **somente dígitos com 10 ou 11 caracteres**. Inválido: pedir de novo (no cadastro) ou **manter o valor anterior** (na edição).
RN03	E-mail: converter para minúsculas; deve conter "@" e ".", **não** conter espaços nem ";".
RN04	Bairro: padronizar com `title()`, remover ";" e usar "Não informado" se vazio.
RN05	Busca, edição e remoção por nome inexistente: avisar "Contato não encontrado." sem erro.
RN06	Os dados devem **permanecer** ao fechar e reabrir o programa.

Exemplo de execução

Trecho de uma execução do programa (o que o usuário digita aparece depois dos dois-pontos de cada pergunta):

Saída esperada

```
=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 1
Nome: maria das dores
Telefone (com DDD): (92) 99123-4567
E-mail: maria@email.com
Bairro: centro
Contato salvo!

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
```

```
0 - Sair
Escolha: 1
Nome:  joão pedro
Telefone (com DDD): 123
Telefone inválido! Use 10 ou 11 dígitos (com DDD).
```

O que entregar

- Arquivo `projeto09_agenda.py` (VS Code) **ou** notebook `Projeto09_Agenda.ipynb` (Colab, com o Drive montado).
- Quadro de **testes** com, no mínimo, 8 casos (incluindo persistência) e **registro de colaboração** da dupla.

Critérios de avaliação

Critério	O que será observado
Funcionamento	Cadastro, listagem, busca, edição, remoção e resumo funcionando.
Persistência	Dados gravados e recarregados; alterações refletidas no arquivo.
Validações	Telefone, e-mail, nome e bairro tratados conforme as regras.
Funções e estruturas	Funções coesas; lista de dicionários bem usada.
Organização e colaboração	Código legível, testes e registro da dupla.

Dicas

- Cada linha do arquivo: `nome;telefone;email;bairro`. Ao ler, use `linha.strip().split(";")`.
- Para limpar o telefone: `for simbolo in " -()": texto = texto.replace(simbolo, "")`.
- Para listar em ordem: monte uma lista só com os nomes, use `.sort()` e busque cada contato pelo nome.
- Crie `buscar_contato(contatos, nome)` e reutilize em cadastro, edição e remoção.
- Para remover: `contatos.remove(contato)` e depois `salvar_contatos(contatos)`.
- Telefone (`isdigit()` e `len()`) e e-mail (`in`) são validações de **formato**.

Desafios extras (opcionais)

- **Exportar CSV:** grave `agenda.csv` com cabeçalho.
- **Telefone repetido:** impeça dois contatos com o mesmo telefone.
- **Listar por bairro:** peça um bairro e mostre só os contatos dele.
- **Backup:** copie a agenda para `agenda_backup.txt` antes de cada remoção.

Projeto 10 – Biblioteca Comunitária

Ficha do projeto	
Tema	Sistema de empréstimos orientado a objetos
Nível	★★★★★
Tempo sugerido	12 a 16 horas
Forma de trabalho	dupla
Aulas relacionadas	Aulas 1 a 19
Conteúdos	classes e objetos que colaboram entre si, dicionário de objetos, arquivos, validações e regras de negócio

Cenário

Uma biblioteca comunitária ribeirinha controla empréstimos em um caderno e vive perdendo livros e prazos. Você vai desenvolver o **sistema da Biblioteca Comunitária**, usando **duas classes que colaboram**: `Livro` (cada exemplar) e `Biblioteca` (o acervo e as operações). O sistema controla **empréstimos, devoluções e multas por atraso**, e guarda o acervo em arquivo.

Este é um projeto de **fechamento do curso**, no nível do Projeto 5. Multas e prazos são **fictícios**.

Objetivos de aprendizagem

Ao concluir este projeto, você será capaz de:

- Modelar o problema com **duas classes que se relacionam**.
- Guardar objetos em um **dicionário** (`codigo → Livro`).
- Aplicar **regras de negócio** (prazo, multa e limite de empréstimos).
- Persistir o acervo em **arquivo** e recarregá-lo.
- Devolver **tuplas** `(sucesso, mensagem)` para separar a lógica da interface.

Dados de partida

Código	Título (acervo inicial)	Autor
L001	Dom Casmurro	Machado de Assis
L002	Macunaíma	Mário de Andrade
L003	A Selva	Ferreira de Castro
L004	Memórias Póstumas de Brás Cubas	Machado de Assis

Requisitos funcionais

Código	Requisito
RF01	Classe **`Livro`** com atributos `codigo`, `titulo`, `autor` e `leitor` (texto vazio = disponível) e métodos **`esta_disponivel()`**, **`emprestar(leitor)`**, **`devolver(dias)`** (devolve a multa) e **`descricao()`**.
RF02	Classe **`Biblioteca`** com o dicionário `livros` e os métodos **`adicionar(titulo, autor)`**, **`buscar(codigo)`**, **`buscar_por_titulo(trecho)`**, **`emprestar(codigo, leitor)`**, **`devolver(codigo, dias)`**, **`salvar()`** e **`carregar()`**.
RF03	Na **primeira execução** (sem arquivo), criar o **acervo inicial** da tabela e salvá-lo em `acervo.txt`; nas seguintes, **carregar** o arquivo.
RF04	**Menu**: 1 – Cadastrar livro, 2 – Listar acervo, 3 – Buscar por título, 4 – Emprestar, 5 – Devolver, 6 – Relatório, 0 – Sair.
RF05	**Emprestar** e **devolver** devolvem uma tupla `(sucesso, mensagem)`, que o menu apenas exibe.
RF06	**Relatório**: total de livros, disponíveis, emprestados e **total de multas** arrecadadas na sessão.
RF07	Salvar o acervo **após cada operação** que o altere (cadastro, empréstimo e devolução).

Regras de negócio

Código	Regra
RN01	O **código** é gerado automaticamente no formato `L001`, `L002`... (sequência do acervo).
RN02	Título e autor **não vazios** e sem ";"; padronize com `strip()` e `title()` no cadastro.
RN03	**Prazo** de empréstimo: **7 dias**. Atraso: **R$ 0,50 por dia** que passar do prazo (devolução em até 7 dias: sem multa).
RN04	Um leitor pode ter **no máximo 3** livros emprestados ao mesmo tempo.
RN05	Não é possível emprestar livro **inexistente** ou **já emprestado** (informar com quem está), nem devolver livro **não emprestado**.
RN06	O nome do leitor é padronizado com `strip()` e `title()` e **não pode ser vazio**. Os dias de uso devem ser **maiores que zero**.
RN07	Busca por título: o trecho **pode estar em qualquer parte** do título, sem diferenciar maiúsculas de minúsculas.

Exemplo de execução

Trecho de uma execução do programa (o que o usuário digita aparece depois dos dois-pontos de cada pergunta):

Saída esperada

```
=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
```

```
Escolha: 2

--- ACERVO ---
L001 | Dom Casmurro | Machado de Assis | Disponível
L002 | Macunaíma | Mário de Andrade | Disponível
L003 | A Selva | Ferreira de Castro | Disponível
L004 | Memórias Póstumas de Brás Cubas | Machado de Assis | Disponível

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 1
Título: vidas secas
Autor: graciliano ramos
```

O que entregar

- Arquivo `projeto10_biblioteca.py` (VS Code) **ou** notebook `Projeto10_Biblioteca.ipynb` (Colab, com o Drive montado).
- Quadro de **testes** com, no mínimo, 10 casos (incluindo persistência) e **registro de colaboração** da dupla.
- **Apresentação oral** de 5 a 7 minutos: demonstração e explicação da relação entre as duas classes.

Critérios de avaliação

Critério	O que será observado
Funcionamento	Cadastro, empréstimo, devolução com multa e relatório funcionando.
POO	Duas classes coerentes que colaboram; métodos devolvem valores adequados.
Persistência	Acervo e empréstimos salvos e recarregados corretamente.
Regras de negócio	RN01 a RN07 implementadas.
Organização, testes e apresentação	Código legível, testes completos e boa comunicação oral.

Dicas

- No `__init__` de `Biblioteca`, use `os.path.exists(arquivo)` para decidir entre carregar e criar o acervo inicial.
- Guarde cada livro em `self.livros[codigo] = livro`; `self.livros.get(codigo)` devolve `None` se não existir.
- Código com zeros à esquerda: `f"L{len(self.livros) + 1:03d}"`.
- Arquivo: `codigo;titulo;autor;leitor` (leitor vazio quando disponível).

- Para contar os empréstimos do leitor, percorra `self.livros.values()`.
- A tupla de retorno: `return False, "Livro indisponível."`, e no menu: `ok, msg = biblioteca.emprestar(codigo, leitor)`.

Desafios extras (opcionais)

- **Multas em arquivo:** persista o total de multas.
- **Classe `Leitor`:** crie uma classe com nome e livros em posse.
- **Reserva:** permita reservar livros emprestados.
- **Mais emprestados:** conte quantas vezes cada livro foi emprestado.

Gabarito Oficial dos Projetos Integradores

> **USO EXCLUSIVO DO PROFESSOR**
> Esta parte contém as soluções de referência. **Remova estas páginas antes de distribuir o material aos alunos.**

Como corrigir

- **Execute o programa do aluno** com os casos de teste da seção “Casos de teste para correção” e compare com o resultado esperado.
- **Aceite soluções diferentes** da de referência, desde que atendam aos requisitos (RF) e às regras (RN) e usem os conteúdos do curso.
- Use a **matriz de correção** de cada projeto (três níveis: pleno, parcial e insuficiente). A atribuição da nota é de competência exclusiva do professor.
- Consulte “Erros frequentes” para orientar o retorno ao aluno.
- As saídas mostradas foram **geradas por execução real** dos programas de referência, com os dados de teste indicados.

Gabarito – Projeto 6: Quiz da Floresta Amazônica

Solução de referência

projeto06.py

```
# Projeto 6 - Quiz da Floresta Amazônica
melhor_pontuacao = 0
partidas = 0

while True:
    print("\n=== QUIZ DA FLORESTA AMAZÔNICA ===")
    print("1 - Jogar")
    print("2 - Regras")
    print("0 - Sair")
    opcao = input("Escolha: ")

    if opcao == "1":
        pontos = 0
        acertos = 0
        seguidos = 0

        for numero in range(1, 6):
            if numero == 1:
                pergunta = "Quais rios formam o Encontro das Águas, em Manaus?"
                a = "Madeira e Purus"
                b = "Negro e Solimões"
                c = "Tapajós e Xingu"
                d = "Juruá e Japurá"
                correta = "B"
            elif numero == 2:
                pergunta = "Qual peixe amazônico pode ultrapassar 2 metros de comprimento?"
                a = "Tambaqui"
                b = "Pirarucu"
                c = "Jaraqui"
                d = "Matrinxã"
                correta = "B"
            elif numero == 3:
                pergunta = "O Festival Folclórico de Parintins reúne quais bois-bumbás?"
                a = "Garantido e Caprichoso"
                b = "Mimoso e Caprichoso"
                c = "Garantido e Mimoso"
                d = "Faceiro e Caprichoso"
                correta = "A"
            elif numero == 4:
                pergunta = "Em qual cidade fica o Teatro Amazonas?"
                a = "Belém"
                b = "Manaus"
                c = "Parintins"
                d = "Santarém"
                correta = "B"
            else:
                pergunta = "Qual é o rio de maior volume de água do mundo?"
                a = "Nilo"
                b = "Mississippi"
                c = "Amazonas"
                d = "Yangtzé"
                correta = "C"

            print(f"\nPergunta {numero} de 5")
            print(pergunta)
            print(f"A) {a}")
            print(f"B) {b}")
            print(f"C) {c}")
```

```
            print(f"D) {d}")

            resposta = input("Sua resposta (A, B, C ou D): ").strip().upper()
            while resposta != "A" and resposta != "B" and resposta != "C" and resposta != "D":
                print("Resposta inválida! Digite A, B, C ou D.")
                resposta = input("Sua resposta (A, B, C ou D): ").strip().upper()

            if resposta == correta:
                pontos += 10
                acertos += 1
                seguidos += 1
                print("Correto! +10 pontos.")
                if seguidos == 3:
                    pontos += 5
                    seguidos = 0
                    print("Bônus de sequência! +5 pontos.")
            else:
                seguidos = 0
                print(f"Errou! A resposta correta era {correta}.")

        print("\n=== RESULTADO ===")
        print(f"Acertos: {acertos} de 5")
        print(f"Pontuação: {pontos}")
        if pontos >= 50:
            titulo = "Guardião da Floresta"
        elif pontos >= 30:
            titulo = "Explorador"
        else:
            titulo = "Aprendiz"
        print(f"Título: {titulo}")

        partidas += 1
        if pontos > melhor_pontuacao:
            melhor_pontuacao = pontos
            print("Novo recorde da sessão!")

    elif opcao == "2":
        print("\n--- REGRAS ---")
        print("São 5 perguntas de múltipla escolha.")
        print("Cada acerto vale 10 pontos.")
        print("A cada 3 acertos seguidos, você ganha 5 pontos de bônus.")
        print("50+ pontos: Guardião da Floresta | 30 a 49: Explorador | abaixo de 30:
Aprendiz.")
    elif opcao == "0":
        print(f"\nPartidas jogadas: {partidas}")
        print(f"Melhor pontuação: {melhor_pontuacao}")
        print("Até a próxima!")
        break
    else:
        print("Opção inválida!")
```

Execução de teste

Teste 1: regras, três partidas (45 pontos, 0 ponto e 55 pontos), resposta e opção inválidas.

Saída obtida

```
=== QUIZ DA FLORESTA AMAZÔNICA ===
1 - Jogar
2 - Regras
0 - Sair
```

```
Escolha: 2

--- REGRAS ---
São 5 perguntas de múltipla escolha.
Cada acerto vale 10 pontos.
A cada 3 acertos seguidos, você ganha 5 pontos de bônus.
50+ pontos: Guardião da Floresta | 30 a 49: Explorador | abaixo de 30: Aprendiz.

=== QUIZ DA FLORESTA AMAZÔNICA ===
1 - Jogar
2 - Regras
0 - Sair
Escolha: 1

Pergunta 1 de 5
Quais rios formam o Encontro das Águas, em Manaus?
A) Madeira e Purus
B) Negro e Solimões
C) Tapajós e Xingu
D) Juruá e Japurá
Sua resposta (A, B, C ou D): B
Correto! +10 pontos.

Pergunta 2 de 5
Qual peixe amazônico pode ultrapassar 2 metros de comprimento?
A) Tambaqui
B) Pirarucu
C) Jaraqui
D) Matrinxã
Sua resposta (A, B, C ou D):  b
Correto! +10 pontos.

Pergunta 3 de 5
O Festival Folclórico de Parintins reúne quais bois-bumbás?
A) Garantido e Caprichoso
B) Mimoso e Caprichoso
C) Garantido e Mimoso
D) Faceiro e Caprichoso
Sua resposta (A, B, C ou D): x
Resposta inválida! Digite A, B, C ou D.
Sua resposta (A, B, C ou D): A
Correto! +10 pontos.
Bônus de sequência! +5 pontos.

Pergunta 4 de 5
Em qual cidade fica o Teatro Amazonas?
A) Belém
B) Manaus
C) Parintins
D) Santarém
Sua resposta (A, B, C ou D): C
Errou! A resposta correta era B.

Pergunta 5 de 5
Qual é o rio de maior volume de água do mundo?
A) Nilo
B) Mississippi
C) Amazonas
D) Yangtzé
Sua resposta (A, B, C ou D): C
Correto! +10 pontos.

=== RESULTADO ===
```

```
Acertos: 4 de 5
Pontuação: 45
Título: Explorador
Novo recorde da sessão!

=== QUIZ DA FLORESTA AMAZÔNICA ===
1 - Jogar
2 - Regras
0 - Sair
Escolha: 1

Pergunta 1 de 5
Quais rios formam o Encontro das Águas, em Manaus?
A) Madeira e Purus
B) Negro e Solimões
C) Tapajós e Xingu
D) Juruá e Japurá
Sua resposta (A, B, C ou D): A
Errou! A resposta correta era B.

Pergunta 2 de 5
Qual peixe amazônico pode ultrapassar 2 metros de comprimento?
A) Tambaqui
B) Pirarucu
C) Jaraqui
D) Matrinxã
Sua resposta (A, B, C ou D): A
Errou! A resposta correta era B.

Pergunta 3 de 5
O Festival Folclórico de Parintins reúne quais bois-bumbás?
A) Garantido e Caprichoso
B) Mimoso e Caprichoso
C) Garantido e Mimoso
D) Faceiro e Caprichoso
Sua resposta (A, B, C ou D): B
Errou! A resposta correta era A.

Pergunta 4 de 5
Em qual cidade fica o Teatro Amazonas?
A) Belém
B) Manaus
C) Parintins
D) Santarém
Sua resposta (A, B, C ou D): A
Errou! A resposta correta era B.

Pergunta 5 de 5
Qual é o rio de maior volume de água do mundo?
A) Nilo
B) Mississippi
C) Amazonas
D) Yangtzé
Sua resposta (A, B, C ou D): A
Errou! A resposta correta era C.

=== RESULTADO ===
Acertos: 0 de 5
Pontuação: 0
Título: Aprendiz

=== QUIZ DA FLORESTA AMAZÔNICA ===
1 - Jogar
```

```
2 - Regras
0 - Sair
Escolha: 1

Pergunta 1 de 5
Quais rios formam o Encontro das Águas, em Manaus?
A) Madeira e Purus
B) Negro e Solimões
C) Tapajós e Xingu
D) Juruá e Japurá
Sua resposta (A, B, C ou D): B
Correto! +10 pontos.

Pergunta 2 de 5
Qual peixe amazônico pode ultrapassar 2 metros de comprimento?
A) Tambaqui
B) Pirarucu
C) Jaraqui
D) Matrinxã
Sua resposta (A, B, C ou D): B
Correto! +10 pontos.

Pergunta 3 de 5
O Festival Folclórico de Parintins reúne quais bois-bumbás?
A) Garantido e Caprichoso
B) Mimoso e Caprichoso
C) Garantido e Mimoso
D) Faceiro e Caprichoso
Sua resposta (A, B, C ou D): A
Correto! +10 pontos.
Bônus de sequência! +5 pontos.

Pergunta 4 de 5
Em qual cidade fica o Teatro Amazonas?
A) Belém
B) Manaus
C) Parintins
D) Santarém
Sua resposta (A, B, C ou D): B
Correto! +10 pontos.

Pergunta 5 de 5
Qual é o rio de maior volume de água do mundo?
A) Nilo
B) Mississippi
C) Amazonas
D) Yangtzé
Sua resposta (A, B, C ou D): C
Correto! +10 pontos.

=== RESULTADO ===
Acertos: 5 de 5
Pontuação: 55
Título: Guardião da Floresta
Novo recorde da sessão!

=== QUIZ DA FLORESTA AMAZÔNICA ===
1 - Jogar
2 - Regras
0 - Sair
Escolha: 7
Opção inválida!
```

```
=== QUIZ DA FLORESTA AMAZÔNICA ===
1 - Jogar
2 - Regras
0 - Sair
Escolha: 0

Partidas jogadas: 3
Melhor pontuação: 55
Até a próxima!
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Opção 2 no menu	Exibe as regras e volta ao menu
Resposta “ b ” (minúscula e com espaços)	Aceita como B
Resposta “x”	“Resposta inválida! Digite A, B, C ou D.” e a mesma pergunta é repetida
Partida 1: B, b, x→A, C, C	4 acertos; 45 pontos (40 + 5 de bônus no 3º acerto seguido); “Explorador”; novo recorde
Partida 2: A, A, B, A, A (todas erradas)	0 acertos; 0 ponto; “Aprendiz”; sem novo recorde
Partida 3: B, B, A, B, C (todas certas)	5 acertos; 55 pontos (50 + 5 de bônus); “Guardião da Floresta”; novo recorde
Opção 7 no menu	“Opção inválida!”
Sair (0)	3 partidas jogadas; melhor pontuação 55

Matriz de correção

Critério	Pleno	Parcial	Insuficiente
Funcionamento	Menu, partidas e encerramento funcionam; resultados conferem nos 8 casos.	Falha em um cenário (ex.: segunda partida herda pontos).	O jogo trava ou não completa uma partida.
Regras de pontuação	Pontos, bônus (a cada 3 seguidos), títulos e recorde corretos.	Bônus ou recorde com falha.	Pontuação incorreta.
Uso dos conteúdos	`for/range`, `if/elif/else`, `while` e acumuladores bem aplicados.	Repete código das perguntas sem usar `for`.	Sem repetição ou sem decisões.
Validação	Resposta e menu tratados conforme RN01 e RN06.	Valida só em parte (ex.: não aceita minúscula).	Sem validação.
Organização e testes	Legível, comentado; quadro com 5+ casos.	Pouco comentado ou poucos testes.	Sem organização ou testes.

Erros frequentes e orientações

- **Não zerar `pontos`, `acertos` e `seguidos`** ao iniciar uma nova partida.
- **Usar `or` na validação** (`resposta != "A" or resposta != "B"` é sempre verdadeiro) em vez de `and`.
- **Não zerar `seguidos` depois do bônus** (o 4º acerto recebe bônus indevido) ou **não zerar após um erro**.
- **Esquecer `.upper()`**: a resposta "b" é considerada errada.
- Usar `range(5)` e exibir "Pergunta 0", ou `range(1, 5)` e perder a última pergunta.
- Comparar `pontos > 50` em vez de `>= 50` no título.

Orientações para os desafios extras

- **Mais perguntas:** `perguntas = [{"texto": ..., "alternativas": [...], "correta": "B"}, ...]` e `for item in perguntas:`.
- **Dificuldade:** `multiplicador = 2` para o modo difícil e `pontos += 10 * multiplicador`.
- **Ajuda:** variável `usou_ajuda = False` por partida; se `resposta == "H"` e `not usou_ajuda`, esconda duas alternativas erradas e debite 5 pontos.

Gabarito – Projeto 7: Meu Orçamento Familiar

Solução de referência

projeto07.py

```
# Projeto 7 - Meu Orçamento Familiar (dados fictícios)
categorias = ["Moradia", "Alimentação", "Transporte", "Saúde", "Lazer", "Outros"]
lancamentos = []

while True:
    print("\n=== MEU ORÇAMENTO FAMILIAR ===")
    print("1 - Registrar receita")
    print("2 - Registrar despesa")
    print("3 - Resumo do mês")
    print("4 - Despesas por categoria")
    print("5 - Listar lançamentos")
    print("0 - Sair")
    opcao = input("Escolha: ")

    if opcao == "1" or opcao == "2":
        descricao = input("Descrição: ").strip().capitalize()
        while descricao == "":
            print("A descrição não pode ficar vazia.")
            descricao = input("Descrição: ").strip().capitalize()

        categoria = "Receita"
        if opcao == "2":
            print("Categorias:")
            for i in range(len(categorias)):
                print(f"{i + 1} - {categorias[i]}")
            numero = int(input("Categoria (número): "))
            while numero < 1 or numero > len(categorias):
                print("Categoria inválida!")
                numero = int(input("Categoria (número): "))
            categoria = categorias[numero - 1]

        valor = float(input("Valor (R$): "))
        while valor <= 0:
            print("O valor deve ser maior que zero.")
            valor = float(input("Valor (R$): "))

        if opcao == "1":
            tipo = "receita"
        else:
            tipo = "despesa"
        lancamentos.append({"tipo": tipo, "descricao": descricao, "categoria": categoria,
"valor": valor})
        print(f"Lançamento registrado: {descricao} - R$ {valor:.2f}")

    elif opcao == "3":
        receitas = 0
        despesas = 0
        for item in lancamentos:
            if item["tipo"] == "receita":
                receitas += item["valor"]
            else:
                despesas += item["valor"]
        saldo = receitas - despesas

        print("\n--- RESUMO DO MÊS ---")
        print(f"Receitas: R$ {receitas:.2f}")
        print(f"Despesas: R$ {despesas:.2f}")
```

```
        print(f"Saldo: R$ {saldo:.2f}")
        if receitas == 0:
            print("Registre ao menos uma receita para analisar o orçamento.")
        else:
            percentual = despesas / receitas * 100
            print(f"Comprometimento da renda: {percentual:.1f}%")
            if percentual <= 70:
                print("Situação: Saudável")
            elif percentual <= 100:
                print("Situação: Atenção - despesas altas")
            else:
                print("Situação: Alerta - despesas maiores que a receita")

    elif opcao == "4":
        totais = {}
        for nome in categorias:
            totais[nome] = 0
        total_despesas = 0
        for item in lancamentos:
            if item["tipo"] == "despesa":
                totais[item["categoria"]] += item["valor"]
                total_despesas += item["valor"]

        if total_despesas == 0:
            print("Nenhuma despesa registrada.")
        else:
            print("\n--- DESPESAS POR CATEGORIA ---")
            maior_categoria = ""
            maior_valor = 0
            for nome, total_cat in totais.items():
                if total_cat > 0:
                    print(f"{nome}: R$ {total_cat:.2f} ({total_cat / total_despesas *
100:.1f}%)")
                if total_cat > maior_valor:
                    maior_valor = total_cat
                    maior_categoria = nome
            print(f"Maior gasto: {maior_categoria} (R$ {maior_valor:.2f})")

    elif opcao == "5":
        if len(lancamentos) == 0:
            print("Nenhum lançamento registrado.")
        else:
            print("\n--- LANÇAMENTOS ---")
            for i in range(len(lancamentos)):
                item = lancamentos[i]
                if item["tipo"] == "receita":
                    sinal = "+"
                else:
                    sinal = "-"
                print(f"{i + 1}. [{item['categoria']}] {item['descricao']}: {sinal} R$
{item['valor']:.2f}")

    elif opcao == "0":
        print("Até logo!")
        break
    else:
        print("Opção inválida!")
```

Execução de teste

Teste 1: cenário completo: sem dados, entradas inválidas, resumo, categorias e listagem.

Saída obtida

```
=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 3

--- RESUMO DO MÊS ---
Receitas: R$ 0.00
Despesas: R$ 0.00
Saldo: R$ 0.00
Registre ao menos uma receita para analisar o orçamento.

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 4
Nenhuma despesa registrada.

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 5
Nenhum lançamento registrado.

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 1
Descrição:   salário
Valor (R$): 3000
Lançamento registrado: Salário - R$ 3000.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 1
Descrição:
A descrição não pode ficar vazia.
Descrição: extra feira
Valor (R$): 450.5
Lançamento registrado: Extra feira - R$ 450.50

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
```

```
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 2
Descrição: aluguel
Categorias:
1 - Moradia
2 - Alimentação
3 - Transporte
4 - Saúde
5 - Lazer
6 - Outros
Categoria (número): 1
Valor (R$): 1200
Lançamento registrado: Aluguel - R$ 1200.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 2
Descrição: feira da semana
Categorias:
1 - Moradia
2 - Alimentação
3 - Transporte
4 - Saúde
5 - Lazer
6 - Outros
Categoria (número): 2
Valor (R$): 0
O valor deve ser maior que zero.
Valor (R$): 650.75
Lançamento registrado: Feira da semana - R$ 650.75

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 2
Descrição: barco para o interior
Categorias:
1 - Moradia
2 - Alimentação
3 - Transporte
4 - Saúde
5 - Lazer
6 - Outros
Categoria (número): 9
Categoria inválida!
Categoria (número): 3
Valor (R$): 380
Lançamento registrado: Barco para o interior - R$ 380.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
```

```
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 2
Descrição: cinema
Categorias:
1 - Moradia
2 - Alimentação
3 - Transporte
4 - Saúde
5 - Lazer
6 - Outros
Categoria (número): 5
Valor (R$): 120
Lançamento registrado: Cinema - R$ 120.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 3

--- RESUMO DO MÊS ---
Receitas: R$ 3450.50
Despesas: R$ 2350.75
Saldo: R$ 1099.75
Comprometimento da renda: 68.1%
Situação: Saudável

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 4

--- DESPESAS POR CATEGORIA ---
Moradia: R$ 1200.00 (51.0%)
Alimentação: R$ 650.75 (27.7%)
Transporte: R$ 380.00 (16.2%)
Lazer: R$ 120.00 (5.1%)
Maior gasto: Moradia (R$ 1200.00)

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 5

--- LANÇAMENTOS ---
1. [Receita] Salário: + R$ 3000.00
2. [Receita] Extra feira: + R$ 450.50
3. [Moradia] Aluguel: - R$ 1200.00
4. [Alimentação] Feira da semana: - R$ 650.75
5. [Transporte] Barco para o interior: - R$ 380.00
```

```
6. [Lazer] Cinema: - R$ 120.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 0
Até logo!
```

Teste 2: despesas maiores que a receita (120%).

Saída obtida

```
=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 1
Descrição: renda
Valor (R$): 1000
Lançamento registrado: Renda - R$ 1000.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 2
Descrição: dívida
Categorias:
1 - Moradia
2 - Alimentação
3 - Transporte
4 - Saúde
5 - Lazer
6 - Outros
Categoria (número): 6
Valor (R$): 1200
Lançamento registrado: Dívida - R$ 1200.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 3

--- RESUMO DO MÊS ---
Receitas: R$ 1000.00
Despesas: R$ 1200.00
Saldo: R$ -200.00
Comprometimento da renda: 120.0%
```

```
Situação: Alerta - despesas maiores que a receita

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 0
Até logo!
```

Teste 3: despesas altas (90%).

Saída obtida

```
=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 1
Descrição: renda
Valor (R$): 1000
Lançamento registrado: Renda - R$ 1000.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 2
Descrição: conta de luz
Categorias:
1 - Moradia
2 - Alimentação
3 - Transporte
4 - Saúde
5 - Lazer
6 - Outros
Categoria (número): 1
Valor (R$): 900
Lançamento registrado: Conta de luz - R$ 900.00

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 3

--- RESUMO DO MÊS ---
Receitas: R$ 1000.00
Despesas: R$ 900.00
Saldo: R$ 100.00
Comprometimento da renda: 90.0%
```

```
Situação: Atenção - despesas altas

=== MEU ORÇAMENTO FAMILIAR ===
1 - Registrar receita
2 - Registrar despesa
3 - Resumo do mês
4 - Despesas por categoria
5 - Listar lançamentos
0 - Sair
Escolha: 0
Até logo!
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Opções 3, 4 e 5 sem nenhum lançamento	Resumo com R$ 0,00 e aviso “Registre ao menos uma receita…”; “Nenhuma despesa registrada.”; “Nenhum lançamento registrado.”
Descrição vazia na receita “extra feira”	“A descrição não pode ficar vazia.” e nova pergunta; salva como “Extra feira”
Valor 0 na feira; categoria 9 no barco	“O valor deve ser maior que zero.”; “Categoria inválida!” e novas perguntas
Receitas 3000 + 450,50; despesas 1200 + 650,75 + 380 + 120	Receitas R$ 3450,50; despesas R$ 2350,75; saldo R$ 1099,75; comprometimento 68,1%; “Saudável”
Despesas por categoria do cenário	Moradia 51,0%; Alimentação 27,7%; Transporte 16,2%; Lazer 5,1%; maior gasto: Moradia (R$ 1200,00)
Renda 1000 e despesa 900	90,0% – “Atenção – despesas altas”
Renda 1000 e despesa 1200	120,0% – “Alerta – despesas maiores que a receita”
Listagem final	6 lançamentos numerados, com sinal + nas receitas e − nas despesas

Matriz de correção

Critério	Pleno	Parcial	Insuficiente
Funcionamento	Todas as opções funcionam e conferem com os casos de teste.	Uma opção com falha (ex.: categorias).	O menu não executa as operações.
Estruturas de dados	Lista de dicionários, lista de categorias e dicionário de totais corretos.	Usa listas, mas sem o dicionário de totais (ou índices frágeis).	Variáveis soltas, sem listas.
Cálculos	Totais, saldo e porcentagens corretos nos 3 níveis.	Erro de arredondamento/limite (ex.: 70% exato).	Cálculos incorretos.
Regras de negócio	RN01 a RN06 implementadas, inclusive sem dados.	Faltam 1 ou 2 validações (ex.: divisão por zero).	Gera erro com dados vazios.
Organização e testes	Legível, comentado; quadro com 6+ casos.	Poucos testes ou pouco comentado.	Sem organização ou testes.

Erros frequentes e orientações

- **Dividir por `receitas` sem testar se é zero** (ZeroDivisionError).
- **Usar `categorias[numero]` em vez de `categorias[numero - 1]`** (a lista começa em 0).
- **Esquecer de zerar o dicionário de totais** a cada consulta, acumulando valores repetidos.
- **Somar receitas nas categorias de despesa** por não filtrar por `tipo`.
- **Comparar `percentual < 70`** quando o limite é inclusivo (`<= 70`).
- Usar o mesmo nome (`valor`) para variáveis diferentes dentro do mesmo laço, sobrescrevendo dados.

Orientações para os desafios extras

- **Gráfico de barras:** `print("#" * int(percentual / 5))` ao lado de cada categoria.
- **Meta de poupança:** `meta = float(input(...))`; `if saldo >= meta: ...`.
- **Remover lançamento:** `numero = int(input(...))`; `lancamentos.pop(numero - 1)` após validar o intervalo.
- **Persistência:** uma linha `tipo;descricao;categoria;valor` por lançamento; ao carregar, converter o valor com `float()`.

Gabarito – Projeto 8: Forca Amazônica

Solução de referência

projeto08.py

```
# Projeto 8 - Forca Amazônica
PALAVRAS = (
    ("tambaqui", "Peixe de escamas, famoso na culinária do Amazonas"),
    ("pirarucu", "Um dos maiores peixes de água doce, pode passar de 2 metros"),
    ("tucunare", "Peixe de rios e lagos, muito procurado na pesca esportiva"),
    ("castanha", "Semente de árvore gigante, também chamada castanha-do-pará"),
    ("cupuacu", "Fruta de polpa cremosa, usada em sucos e doces"),
    ("jaraqui", "Peixe pequeno e muito consumido pelos manauaras"),
)
MAX_ERROS = 6


def escolher_palavra(numero_rodada):
    posicao = (numero_rodada - 1) % len(PALAVRAS)
    return PALAVRAS[posicao]


def montar_exibicao(palavra, acertadas):
    exibicao = ""
    for letra in palavra:
        if letra in acertadas:
            exibicao += letra + " "
        else:
            exibicao += "_ "
    return exibicao.strip()


def venceu(palavra, acertadas):
    for letra in palavra:
        if letra not in acertadas:
            return False
    return True


def ler_letra(tentadas):
    while True:
        entrada = input("Digite uma letra: ").strip().lower()
        if len(entrada) != 1 or not entrada.isalpha():
            print("Digite apenas UMA letra (de a a z).")
        elif entrada in tentadas:
            print("Você já tentou essa letra!")
        else:
            return entrada


def jogar_rodada(numero_rodada):
    palavra, dica = escolher_palavra(numero_rodada)
    acertadas = []
    erradas = []
    print(f"\n=== RODADA {numero_rodada} ===")
    print(f"Dica: {dica}")

    while len(erradas) < MAX_ERROS and not venceu(palavra, acertadas):
        vidas = MAX_ERROS - len(erradas)
        print(f"\nPalavra: {montar_exibicao(palavra, acertadas)}")
```

```
        print("Vidas: " + "O " * vidas + "X " * len(erradas))
        if len(erradas) > 0:
            print(f"Letras erradas: {erradas}")
        letra = ler_letra(acertadas + erradas)
        if letra in palavra:
            acertadas.append(letra)
            print("Acertou!")
        else:
            erradas.append(letra)
            print("Errou!")

    if venceu(palavra, acertadas):
        vidas = MAX_ERROS - len(erradas)
        pontos = 10 + vidas * 5
        print(f"\nParabéns! A palavra era {palavra.upper()}. +{pontos} pontos.")
        return True, pontos
    print(f"\nFim de jogo! A palavra era {palavra.upper()}.")
    return False, 0


def principal():
    rodada = 1
    total = 0
    vitorias = 0
    continuar = "s"
    while continuar == "s":
        ganhou, pontos = jogar_rodada(rodada)
        total += pontos
        if ganhou:
            vitorias += 1
        print(f"Placar: {total} pontos | Vitórias: {vitorias} de {rodada} rodada(s)")
        rodada += 1
        continuar = input("Jogar outra rodada? (s/n): ").strip().lower()

    print("\n=== FIM DA SESSÃO ===")
    print(f"Rodadas: {rodada - 1}")
    print(f"Vitórias: {vitorias}")
    print(f"Pontuação total: {total}")


principal()
```

Execução de teste

Teste 1: três rodadas: vitória com 1 erro, derrota, vitória sem erros; letra repetida e entradas inválidas.

Saída obtida

```
=== RODADA 1 ===
Dica: Peixe de escamas, famoso na culinária do Amazonas

Palavra: _ _ _ _ _ _ _ _
Vidas: O O O O O O
Digite uma letra: t
Acertou!

Palavra: t _ _ _ _ _ _ _
Vidas: O O O O O O
```

```
Digite uma letra: x
Errou!

Palavra: t _ _ _ _ _ _ _
Vidas: O O O O O X
Letras erradas: ['x']
Digite uma letra: t
Você já tentou essa letra!
Digite uma letra: 12
Digite apenas UMA letra (de a a z).
Digite uma letra: ab
Digite apenas UMA letra (de a a z).
Digite uma letra: a
Acertou!

Palavra: t a _ _ a _ _ _
Vidas: O O O O O X
Letras erradas: ['x']
Digite uma letra: m
Acertou!

Palavra: t a m _ a _ _ _
Vidas: O O O O O X
Letras erradas: ['x']
Digite uma letra: b
Acertou!

Palavra: t a m b a _ _ _
Vidas: O O O O O X
Letras erradas: ['x']
Digite uma letra: q
Acertou!

Palavra: t a m b a q _ _
Vidas: O O O O O X
Letras erradas: ['x']
Digite uma letra: u
Acertou!

Palavra: t a m b a q u _
Vidas: O O O O O X
Letras erradas: ['x']
Digite uma letra: i
Acertou!

Parabéns! A palavra era TAMBAQUI. +35 pontos.
Placar: 35 pontos | Vitórias: 1 de 1 rodada(s)
Jogar outra rodada? (s/n): s

=== RODADA 2 ===
Dica: Um dos maiores peixes de água doce, pode passar de 2 metros

Palavra: _ _ _ _ _ _ _ _
Vidas: O O O O O O
Digite uma letra: z
Errou!

Palavra: _ _ _ _ _ _ _ _
Vidas: O O O O O X
Letras erradas: ['z']
Digite uma letra: x
Errou!
```

```
Palavra: _ _ _ _ _ _ _ _
Vidas: O O O O X X
Letras erradas: ['z', 'x']
Digite uma letra: w
Errou!

Palavra: _ _ _ _ _ _ _ _
Vidas: O O O X X X
Letras erradas: ['z', 'x', 'w']
Digite uma letra: k
Errou!

Palavra: _ _ _ _ _ _ _ _
Vidas: O O X X X X
Letras erradas: ['z', 'x', 'w', 'k']
Digite uma letra: j
Errou!

Palavra: _ _ _ _ _ _ _ _
Vidas: O X X X X X
Letras erradas: ['z', 'x', 'w', 'k', 'j']
Digite uma letra: h
Errou!

Fim de jogo! A palavra era PIRARUCU.
Placar: 35 pontos | Vitórias: 1 de 2 rodada(s)
Jogar outra rodada? (s/n): s

=== RODADA 3 ===
Dica: Peixe de rios e lagos, muito procurado na pesca esportiva

Palavra: _ _ _ _ _ _ _ _
Vidas: O O O O O O
Digite uma letra: t
Acertou!

Palavra: t _ _ _ _ _ _ _
Vidas: O O O O O O
Digite uma letra: u
Acertou!

Palavra: t u _ u _ _ _ _
Vidas: O O O O O O
Digite uma letra: c
Acertou!

Palavra: t u c u _ _ _ _
Vidas: O O O O O O
Digite uma letra: n
Acertou!

Palavra: t u c u n _ _ _
Vidas: O O O O O O
Digite uma letra: a
Acertou!

Palavra: t u c u n a _ _
Vidas: O O O O O O
Digite uma letra: r
Acertou!
```

```
Palavra: t u c u n a r _
Vidas: O O O O O O
Digite uma letra: e
Acertou!

Parabéns! A palavra era TUCUNARE. +40 pontos.
Placar: 75 pontos | Vitórias: 2 de 3 rodada(s)
Jogar outra rodada? (s/n): n

=== FIM DA SESSÃO ===
Rodadas: 3
Vitórias: 2
Pontuação total: 75
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Rodada 1 (tambaqui): t, x, t, “12”, “ab”, a, m, b, q, u, i	“x” erra (1 vida perdida); “t” repetida: aviso sem perder vida; “12” e “ab” inválidas; vitória com 5 vidas = **35 pontos**
Rodada 2 (pirarucu): z, x, w, k, j, h	Seis erros: derrota, revela PIRARUCU, **0 ponto**
Rodada 3 (tucunare): t, u, c, n, a, r, e	Vitória sem erros (6 vidas) = **40 pontos**
Placar ao final	3 rodadas; 2 vitórias; **75 pontos**
`montar_exibicao("tambaqui", ["t", "a"])`	`t a _ _ a _ _ _`
`venceu("cupuacu", ["c", "u", "p", "a"])`	`True`
Rodada 7	Volta à palavra da rodada 1 (tambaqui)

Matriz de correção

Critério	Pleno	Parcial	Insuficiente
Funcionamento	Rodadas de vitória e derrota e placar corretos nos casos de teste.	Uma situação com falha (ex.: letra repetida desconta vida).	O jogo não completa uma rodada.
Modularização	As 6 funções exigidas, com parâmetros e `return` corretos.	Funções criadas, mas com lógica misturada ou variáveis globais.	Programa sem funções.
Strings e listas	Letras certas/erradas, exibição e validação corretas.	Exibição ou lista de tentadas com falhas.	Sem controle das letras.
Regras de negócio	RN01 a RN06 implementadas, inclusive pontuação.	Faltam 1 ou 2 regras.	Pontuação ou vidas incorretas.
Organização e colaboração	Legível, comentado, testes e registro da dupla completos.	Falta registro ou testes.	Sem organização ou colaboração.

Erros frequentes e orientações

- **Descontar vida de letra repetida** por não verificar a lista de tentadas.
- **Esquecer `.lower()`**: “A” e “a” são tratados como letras diferentes.

- **Não validar o tamanho** (aceitar “ab” ou texto vazio) ou aceitar números e símbolos (`isalpha`).
- **Esquecer o `return` em `venceu`** ou colocar `return True` dentro do `for` (a função encerra na primeira letra).
- **Usar `acertadas` global** em vez de recebê-la por parâmetro: a lista “vaza” entre rodadas.
- **Posição incorreta na rodada**: sem `% len(PALAVRAS)` ocorre IndexError na 7ª rodada.

Orientações para os desafios extras

- **Desenho da forca:** `ESTAGIOS = ("...", "...", ...)` com 7 textos e `print(ESTAGIOS[len(erradas)])`.
- **Palavras sorteadas:** `import random` e `palavra, dica = random.choice(PALAVRAS)`.
- **Placar em arquivo:** ler o recorde no início (`try/except FileNotFoundError`), comparar e regravar com modo `"w"`.
- **Temas:** `PALAVRAS = {"Peixes": (...), "Frutas": (...)}` e menu para escolher o tema.

Gabarito – Projeto 9: Agenda de Contatos Comunitária

Solução de referência

projeto09.py

```
# Projeto 9 - Agenda de Contatos Comunitária (dados fictícios)
ARQUIVO = "agenda.txt"



def carregar_contatos():
    contatos = []
    try:
        with open(ARQUIVO, "r", encoding="utf-8") as f:
            for linha in f:
                nome, telefone, email, bairro = linha.strip().split(";")
                contatos.append({"nome": nome, "telefone": telefone, "email": email, "bairro":
bairro})
    except FileNotFoundError:
        pass
    return contatos



def salvar_contatos(contatos):
    with open(ARQUIVO, "w", encoding="utf-8") as f:
        for c in contatos:
            f.write(f"{c['nome']};{c['telefone']};{c['email']};{c['bairro']}\n")



def limpar_telefone(texto):
    for simbolo in " -()":
        texto = texto.replace(simbolo, "")
    return texto



def telefone_valido(telefone):
    return telefone.isdigit() and (len(telefone) == 10 or len(telefone) == 11)



def email_valido(email):
    return "@" in email and "." in email and " " not in email and ";" not in email



def buscar_contato(contatos, nome):
    for c in contatos:
        if c["nome"] == nome:
            return c
    return None



def buscar_por_trecho(contatos, trecho):
    encontrados = []
    for c in contatos:
        if trecho.strip().lower() in c["nome"].lower():
            encontrados.append(c)
    return encontrados



def formatar(c):
```

```
    return f"{c['nome']} | {c['telefone']} | {c['email']} | {c['bairro']}"


def cadastrar_contato(contatos):
    nome = input("Nome: ").strip().title()
    if nome == "" or ";" in nome:
        print("Nome inválido.")
        return
    if buscar_contato(contatos, nome) is not None:
        print("Já existe um contato com esse nome.")
        return

    telefone = limpar_telefone(input("Telefone (com DDD): "))
    while not telefone_valido(telefone):
        print("Telefone inválido! Use 10 ou 11 dígitos (com DDD).")
        telefone = limpar_telefone(input("Telefone (com DDD): "))

    email = input("E-mail: ").strip().lower()
    while not email_valido(email):
        print("E-mail inválido!")
        email = input("E-mail: ").strip().lower()

    bairro = input("Bairro: ").strip().title().replace(";", "")
    if bairro == "":
        bairro = "Não informado"

    contatos.append({"nome": nome, "telefone": telefone, "email": email, "bairro": bairro})
    salvar_contatos(contatos)
    print("Contato salvo!")


def listar_contatos(contatos):
    if len(contatos) == 0:
        print("Agenda vazia.")
        return
    nomes = []
    for c in contatos:
        nomes.append(c["nome"])
    nomes.sort()
    print(f"\n--- AGENDA ({len(nomes)} contatos) ---")
    for nome in nomes:
        print(formatar(buscar_contato(contatos, nome)))


def editar_contato(contatos):
    nome = input("Nome do contato a editar: ").strip().title()
    contato = buscar_contato(contatos, nome)
    if contato is None:
        print("Contato não encontrado.")
        return
    print("Pressione Enter para manter o valor atual.")

    novo = limpar_telefone(input(f"Telefone [{contato['telefone']}]: "))
    if novo != "":
        if telefone_valido(novo):
            contato["telefone"] = novo
        else:
            print("Telefone inválido: valor anterior mantido.")

    novo = input(f"E-mail [{contato['email']}]: ").strip().lower()
    if novo != "":
        if email_valido(novo):
```

```
            contato["email"] = novo
        else:
            print("E-mail inválido: valor anterior mantido.")

    novo = input(f"Bairro [{contato['bairro']}]: ").strip().title().replace(";", "")
    if novo != "":
        contato["bairro"] = novo

    salvar_contatos(contatos)
    print("Contato atualizado!")



def remover_contato(contatos):
    nome = input("Nome do contato a remover: ").strip().title()
    contato = buscar_contato(contatos, nome)
    if contato is None:
        print("Contato não encontrado.")
        return
    confirma = input(f"Remover {nome}? (s/n): ").strip().lower()
    if confirma == "s":
        contatos.remove(contato)
        salvar_contatos(contatos)
        print("Contato removido.")
    else:
        print("Remoção cancelada.")



def resumo_por_bairro(contatos):
    if len(contatos) == 0:
        print("Agenda vazia.")
        return
    contagem = {}
    for c in contatos:
        bairro = c["bairro"]
        if bairro in contagem:
            contagem[bairro] += 1
        else:
            contagem[bairro] = 1
    print("\n--- CONTATOS POR BAIRRO ---")
    for bairro, qtd in contagem.items():
        print(f"{bairro}: {qtd}")



contatos = carregar_contatos()

while True:
    print("\n=== AGENDA COMUNITÁRIA ===")
    print("1 - Cadastrar")
    print("2 - Listar")
    print("3 - Buscar por trecho do nome")
    print("4 - Editar")
    print("5 - Remover")
    print("6 - Resumo por bairro")
    print("0 - Sair")
    opcao = input("Escolha: ")

    if opcao == "1":
        cadastrar_contato(contatos)
    elif opcao == "2":
        listar_contatos(contatos)
    elif opcao == "3":
        achados = buscar_por_trecho(contatos, input("Trecho do nome: "))
        if len(achados) == 0:
```

```
            print("Nenhum contato encontrado.")
        else:
            for c in achados:
                print(formatar(c))
    elif opcao == "4":
        editar_contato(contatos)
    elif opcao == "5":
        remover_contato(contatos)
    elif opcao == "6":
        resumo_por_bairro(contatos)
    elif opcao == "0":
        print("Até logo!")
        break
    else:
        print("Opção inválida!")
```

Execução de teste

Teste 1: 1ª execução: cadastros e validações, listagem, busca, edição, remoção e resumo.

Saída obtida

```
=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 1
Nome: maria das dores
Telefone (com DDD): (92) 99123-4567
E-mail: maria@email.com
Bairro: centro
Contato salvo!

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 1
Nome:  joão pedro
Telefone (com DDD): 123
Telefone inválido! Use 10 ou 11 dígitos (com DDD).
Telefone (com DDD): 92 3234-5678
E-mail: joaoemail.com
E-mail inválido!
E-mail: joao@email.com
Bairro: compensa
Contato salvo!

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
```

```
6 - Resumo por bairro
0 - Sair
Escolha: 1
Nome: MARIA DAS DORES
Já existe um contato com esse nome.

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 1
Nome: ana lima
Telefone (com DDD): 92988887777
E-mail: ana@email.com
Bairro:
Contato salvo!

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 2

--- AGENDA (3 contatos) ---
Ana Lima | 92988887777 | ana@email.com | Não informado
João Pedro | 9232345678 | joao@email.com | Compensa
Maria Das Dores | 92991234567 | maria@email.com | Centro

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 3
Trecho do nome: MAR
Maria Das Dores | 92991234567 | maria@email.com | Centro

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 3
Trecho do nome: xyz
Nenhum contato encontrado.

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
```

```
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 4
Nome do contato a editar: joão pedro
Pressione Enter para manter o valor atual.
Telefone [9232345678]:
E-mail [joao@email.com]: joao.novo@email.com
Bairro [Compensa]:
Contato atualizado!

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 4
Nome do contato a editar: xingu
Contato não encontrado.

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 5
Nome do contato a remover: ana lima
Remover Ana Lima? (s/n): n
Remoção cancelada.

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 5
Nome do contato a remover: ana lima
Remover Ana Lima? (s/n): s
Contato removido.

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 6

--- CONTATOS POR BAIRRO ---
Centro: 1
Compensa: 1
```

```
=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 9
Opção inválida!

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 0
Até logo!
```

Teste 2: 2ª execução: os contatos foram carregados do arquivo (persistência).

Saída obtida

```
=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 2

--- AGENDA (2 contatos) ---
João Pedro | 9232345678 | joao.novo@email.com | Compensa
Maria Das Dores | 92991234567 | maria@email.com | Centro

=== AGENDA COMUNITÁRIA ===
1 - Cadastrar
2 - Listar
3 - Buscar por trecho do nome
4 - Editar
5 - Remover
6 - Resumo por bairro
0 - Sair
Escolha: 0
Até logo!
```

Conteúdo do arquivo `agenda.txt` ao final

Arquivo gerado

```
Maria Das Dores;92991234567;maria@email.com;Centro
João Pedro;9232345678;joao.novo@email.com;Compensa
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Cadastrar “maria das dores”, “(92) 99123-4567”	Telefone salvo como 92991234567; nome “Maria Das Dores”
Cadastrar “ joão pedro ” com telefone “123” e e-mail “joaoemail.com”	“Telefone inválido!” e “E-mail inválido!”, com novas perguntas; salva com “92 3234-5678” → 9232345678
Cadastrar “MARIA DAS DORES” novamente	“Já existe um contato com esse nome.”
Cadastrar “ana lima” sem bairro	Bairro “Não informado”
Listar	Ordem alfabética: Ana Lima, João Pedro, Maria Das Dores
Buscar “MAR” e “xyz”	Encontra Maria Das Dores; “Nenhum contato encontrado.”
Editar João Pedro (Enter, novo e-mail, Enter)	Só o e-mail muda para joao.novo@email.com; “Contato atualizado!”
Editar “xingu”	“Contato não encontrado.”
Remover Ana Lima com “n” e depois com “s”	“Remoção cancelada.”; depois “Contato removido.”
Resumo por bairro	Centro: 1; Compensa: 1
Fechar e reabrir	Maria Das Dores e João Pedro continuam na agenda

Matriz de correção

Critério	Pleno	Parcial	Insuficiente
Funcionamento	Todas as operações do CRUD funcionam e conferem com os casos de teste.	Uma operação com falha (ex.: edição ou remoção).	O menu não executa as operações.
Persistência	Carrega e salva corretamente após cada alteração; trata arquivo inexistente.	Salva, mas não carrega (ou não salva após editar/remover).	Dados perdidos ao fechar.
Validações	RN01 a RN04 implementadas (nome, telefone, e-mail, bairro).	Faltam 1 ou 2 validações.	Aceita dados inválidos.
Funções e estruturas	Funções coesas e reaproveitadas (ex.: `buscar_contato`).	Funções com código repetido.	Sem funções.
Organização e colaboração	Legível, comentado, testes e registro da dupla completos.	Falta registro ou testes.	Sem organização ou colaboração.

Erros frequentes e orientações

- **Editar o contato mas esquecer de chamar `salvar_contatos`**: a alteração some ao reabrir.
- **Remover um item da lista dentro de um `for` sobre ela mesma** (pula elementos).
- **Comparar o trecho sem padronizar** (`"mar" in "Maria"` é falso por causa da maiúscula).
- **Não limpar o telefone** antes de validar: “(92) 99123-4567” é considerado inválido.

- **Aceitar ";" nos dados**: a leitura do arquivo quebra com `ValueError`.
- **Esquecer o `.strip()`** ao ler o arquivo, deixando o `\n` no último campo.
- **Usar o modo `"a"` ao salvar tudo**: duplica todos os contatos a cada gravação.

Orientações para os desafios extras

- **Exportar CSV:** gravar a linha de cabeçalho `nome,telefone,email,bairro` e uma linha por contato com `f.write(f"{c['nome']},...\n")`.
- **Telefone repetido:** percorrer a lista comparando `c["telefone"] == telefone` antes de salvar.
- **Listar por bairro:** `for c in contatos: if c["bairro"] == bairro: print(formatar(c))`.
- **Backup:** ler `agenda.txt` e gravar o mesmo conteúdo em `agenda_backup.txt` (modo `"w"`).

Gabarito – Projeto 10: Biblioteca Comunitária

Solução de referência

projeto10.py

```
# Projeto 10 - Biblioteca Comunitária (valores fictícios)
import os

ARQUIVO = "acervo.txt"
PRAZO_DIAS = 7
MULTA_POR_DIA = 0.50
LIMITE_EMPRESTIMOS = 3



class Livro:
    def __init__(self, codigo, titulo, autor, leitor=""):
        self.codigo = codigo
        self.titulo = titulo
        self.autor = autor
        self.leitor = leitor

    def esta_disponivel(self):
        return self.leitor == ""

    def emprestar(self, leitor):
        if not self.esta_disponivel():
            return False
        self.leitor = leitor
        return True

    def devolver(self, dias):
        multa = 0.0
        if dias > PRAZO_DIAS:
            multa = (dias - PRAZO_DIAS) * MULTA_POR_DIA
        self.leitor = ""
        return multa

    def descricao(self):
        if self.esta_disponivel():
            situacao = "Disponível"
        else:
            situacao = f"Emprestado a {self.leitor}"
        return f"{self.codigo} | {self.titulo} | {self.autor} | {situacao}"



class Biblioteca:
    def __init__(self, arquivo=ARQUIVO):
        self.arquivo = arquivo
        self.livros = {}
        self.multas = 0.0
        if os.path.exists(self.arquivo):
            self.carregar()
        else:
            self.criar_acervo_inicial()

    def criar_acervo_inicial(self):
        iniciais = [
            ("Dom Casmurro", "Machado de Assis"),
            ("Macunaíma", "Mário de Andrade"),
            ("A Selva", "Ferreira de Castro"),
            ("Memórias Póstumas de Brás Cubas", "Machado de Assis"),
```

```
        ]
        for titulo, autor in iniciais:
            self.adicionar(titulo, autor)

    def adicionar(self, titulo, autor):
        codigo = f"L{len(self.livros) + 1:03d}"
        livro = Livro(codigo, titulo, autor)
        self.livros[codigo] = livro
        self.salvar()
        return livro

    def salvar(self):
        with open(self.arquivo, "w", encoding="utf-8") as f:
            for livro in self.livros.values():
                f.write(f"{livro.codigo};{livro.titulo};{livro.autor};{livro.leitor}\n")

    def carregar(self):
        with open(self.arquivo, "r", encoding="utf-8") as f:
            for linha in f:
                codigo, titulo, autor, leitor = linha.strip().split(";")
                self.livros[codigo] = Livro(codigo, titulo, autor, leitor)

    def buscar(self, codigo):
        return self.livros.get(codigo)

    def buscar_por_titulo(self, trecho):
        achados = []
        for livro in self.livros.values():
            if trecho.strip().lower() in livro.titulo.lower():
                achados.append(livro)
        return achados

    def emprestimos_do_leitor(self, leitor):
        total = 0
        for livro in self.livros.values():
            if livro.leitor == leitor:
                total += 1
        return total

    def emprestar(self, codigo, leitor):
        livro = self.buscar(codigo)
        if livro is None:
            return False, "Código não encontrado."
        if not livro.esta_disponivel():
            return False, f"Livro indisponível (com {livro.leitor})."
        if self.emprestimos_do_leitor(leitor) >= LIMITE_EMPRESTIMOS:
            return False, f"Limite de {LIMITE_EMPRESTIMOS} empréstimos atingido."
        livro.emprestar(leitor)
        self.salvar()
        return True, f"Empréstimo registrado: {livro.titulo} para {leitor}. Prazo:
{PRAZO_DIAS} dias."

    def devolver(self, codigo, dias):
        livro = self.buscar(codigo)
        if livro is None:
            return False, "Código não encontrado."
        if livro.esta_disponivel():
            return False, "Este livro não está emprestado."
        multa = livro.devolver(dias)
        self.multas += multa
        self.salvar()
        if multa > 0:
```

```
            return True, f"Devolvido com atraso de {dias - PRAZO_DIAS} dia(s). Multa: R$
{multa:.2f}."
        return True, "Devolução registrada sem multa."

    def relatorio(self):
        total = len(self.livros)
        emprestados = 0
        for livro in self.livros.values():
            if not livro.esta_disponivel():
                emprestados += 1
        print("\n--- RELATÓRIO ---")
        print(f"Livros no acervo: {total}")
        print(f"Disponíveis: {total - emprestados}")
        print(f"Emprestados: {emprestados}")
        print(f"Multas arrecadadas na sessão: R$ {self.multas:.2f}")


def principal():
    biblioteca = Biblioteca()
    while True:
        print("\n=== BIBLIOTECA COMUNITÁRIA ===")
        print("1 - Cadastrar livro")
        print("2 - Listar acervo")
        print("3 - Buscar por título")
        print("4 - Emprestar")
        print("5 - Devolver")
        print("6 - Relatório")
        print("0 - Sair")
        opcao = input("Escolha: ")

        if opcao == "1":
            titulo = input("Título: ").strip().title()
            if titulo == "" or ";" in titulo:
                print("Título inválido.")
                continue
            autor = input("Autor: ").strip().title()
            if autor == "" or ";" in autor:
                print("Autor inválido.")
                continue
            livro = biblioteca.adicionar(titulo, autor)
            print(f"Livro cadastrado com o código {livro.codigo}.")
        elif opcao == "2":
            print("\n--- ACERVO ---")
            for livro in biblioteca.livros.values():
                print(livro.descricao())
        elif opcao == "3":
            achados = biblioteca.buscar_por_titulo(input("Trecho do título: "))
            if len(achados) == 0:
                print("Nenhum livro encontrado.")
            for livro in achados:
                print(livro.descricao())
        elif opcao == "4":
            codigo = input("Código do livro: ").strip().upper()
            leitor = input("Nome do leitor: ").strip().title()
            if leitor == "":
                print("O nome do leitor é obrigatório.")
                continue
            ok, mensagem = biblioteca.emprestar(codigo, leitor)
            print(mensagem)
        elif opcao == "5":
            codigo = input("Código do livro: ").strip().upper()
            dias = int(input("Dias com o livro: "))
            while dias <= 0:
                print("Informe um número de dias maior que zero.")
                dias = int(input("Dias com o livro: "))
```

```
            ok, mensagem = biblioteca.devolver(codigo, dias)
            print(mensagem)
        elif opcao == "6":
            biblioteca.relatorio()
        elif opcao == "0":
            print("Até logo!")
            break
        else:
            print("Opção inválida!")


principal()
```

Execução de teste

Teste 1: 1ª execução: acervo inicial, cadastro, busca, empréstimos (limite), devoluções (multa) e relatório.

Saída obtida

```
=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 2

--- ACERVO ---
L001 | Dom Casmurro | Machado de Assis | Disponível
L002 | Macunaíma | Mário de Andrade | Disponível
L003 | A Selva | Ferreira de Castro | Disponível
L004 | Memórias Póstumas de Brás Cubas | Machado de Assis | Disponível

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 1
Título: vidas secas
Autor: graciliano ramos
Livro cadastrado com o código L005.

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 1
Título:
Título inválido.
```

```
=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 3
Trecho do título: casmurro
L001 | Dom Casmurro | Machado de Assis | Disponível

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 3
Trecho do título: xyz
Nenhum livro encontrado.

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 4
Código do livro: L001
Nome do leitor: marina
Empréstimo registrado: Dom Casmurro para Marina. Prazo: 7 dias.

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 4
Código do livro: L001
Nome do leitor: joão
Livro indisponível (com Marina).

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 4
Código do livro: L099
Nome do leitor: fulano
Código não encontrado.
```

```
=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 4
Código do livro: L002
Nome do leitor: marina
Empréstimo registrado: Macunaíma para Marina. Prazo: 7 dias.

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 4
Código do livro: L003
Nome do leitor: marina
Empréstimo registrado: A Selva para Marina. Prazo: 7 dias.

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 4
Código do livro: L004
Nome do leitor: marina
Limite de 3 empréstimos atingido.

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 5
Código do livro: L001
Dias com o livro: 10
Devolvido com atraso de 3 dia(s). Multa: R$ 1.50.

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 5
Código do livro: L001
Dias com o livro: 3
Este livro não está emprestado.
```

```
=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 5
Código do livro: L002
Dias com o livro: 5
Devolução registrada sem multa.

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 6

--- RELATÓRIO ---
Livros no acervo: 5
Disponíveis: 4
Emprestados: 1
Multas arrecadadas na sessão: R$ 1.50

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 9
Opção inválida!

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 0
Até logo!
```

Teste 2: 2ª execução: o acervo e os empréstimos foram carregados do arquivo (persistência).

Saída obtida

```
=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
```

```
0 - Sair
Escolha: 2

--- ACERVO ---
L001 | Dom Casmurro | Machado de Assis | Disponível
L002 | Macunaíma | Mário de Andrade | Disponível
L003 | A Selva | Ferreira de Castro | Emprestado a Marina
L004 | Memórias Póstumas de Brás Cubas | Machado de Assis | Disponível
L005 | Vidas Secas | Graciliano Ramos | Disponível

=== BIBLIOTECA COMUNITÁRIA ===
1 - Cadastrar livro
2 - Listar acervo
3 - Buscar por título
4 - Emprestar
5 - Devolver
6 - Relatório
0 - Sair
Escolha: 0
Até logo!
```

Conteúdo do arquivo `acervo.txt` ao final

Arquivo gerado

```
L001;Dom Casmurro;Machado de Assis;
L002;Macunaíma;Mário de Andrade;
L003;A Selva;Ferreira de Castro;Marina
L004;Memórias Póstumas de Brás Cubas;Machado de Assis;
L005;Vidas Secas;Graciliano Ramos;
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Primeira execução, opção 2	Acervo inicial com 4 livros (L001 a L004), todos disponíveis
Cadastrar “vidas secas”, “graciliano ramos”	Cadastrado como L005 (“Vidas Secas”, “Graciliano Ramos”)
Cadastrar com título vazio	“Título inválido.” (nada é cadastrado)
Buscar “casmurro” e “xyz”	Encontra o L001; “Nenhum livro encontrado.”
Emprestar L001 para “marina”; depois L001 para “joão”	Registrado; “Livro indisponível (com Marina).”
Emprestar L099	“Código não encontrado.”
Marina pega L002 e L003; depois tenta L004	Os dois primeiros ok; “Limite de 3 empréstimos atingido.”
Devolver L001 com 10 dias	Atraso de 3 dias; multa R$ 1,50
Devolver L001 de novo (3 dias)	“Este livro não está emprestado.”
Devolver L002 com 5 dias	“Devolução registrada sem multa.”
Relatório	5 livros; 4 disponíveis; 1 emprestado (L003); multas R$ 1,50
Fechar e reabrir	L003 continua “Emprestado a Marina”; L005 continua no acervo

Matriz de correção

Critério	Pleno	Parcial	Insuficiente
Funcionamento	Todas as operações corretas nos casos de teste, inclusive multa e limite.	Uma operação com falha (ex.: multa ou limite).	Não conclui empréstimo ou devolução.
POO	Duas classes coerentes; métodos devolvem valores; menu apenas exibe mensagens.	Usa classes, mas com lógica de regra no menu ou atributos mal definidos.	Sem classes, ou classes sem uso real.
Persistência	Acervo e empréstimos salvos e recarregados; acervo inicial só na primeira execução.	Salva, mas perde o leitor (ou recria o acervo a cada execução).	Dados perdidos ao fechar.
Regras de negócio	RN01 a RN07 implementadas.	Faltam 1 ou 2 regras (ex.: limite de empréstimos).	Multa ou prazo incorretos.
Organização, testes e apresentação	Legível, 10+ casos de teste, apresentação clara.	Poucos testes ou explicação incompleta.	Sem testes ou sem domínio do código.

Erros frequentes e orientações

- **Esquecer o `self`** nos métodos ou nos atributos (`titulo = titulo` em vez de `self.titulo = titulo`).
- **Multar no prazo**: usar `dias >= PRAZO_DIAS` em vez de `dias > PRAZO_DIAS`.
- **Esquecer de limpar o leitor** ao devolver, deixando o livro "emprestado".
- **Gravar o campo `leitor` ausente**: a linha fica com 3 campos e a leitura falha; grave sempre os 4 campos.
- **Criar o acervo inicial a cada execução**, duplicando os livros.
- **Imprimir dentro das classes e de novo no menu** (mensagens duplicadas): os métodos devem **devolver** a mensagem.
- **Contar empréstimos do leitor** com nome diferente por não padronizar (`marina` ≠ `Marina`).

Orientações para os desafios extras

- **Multas em arquivo:** gravar `multas.txt` com o total e somar ao valor lido ao iniciar.
- **Classe `Leitor`:** atributos `nome` e `livros` (lista de códigos), com métodos `pegar` e `devolver`; `Biblioteca` guarda um dicionário de leitores.
- **Reserva:** atributo `reservado_por` em `Livro`; ao devolver, avisar quem reservou.
- **Mais emprestados:** atributo `vezes_emprestado` em `Livro` (incrementado em `emprestar`) e relatório com o maior valor.